

Applying ML Models for Financial Fraud Detection

María Bautista

Report Credit card fraud detection

CF969-7-SP-CO

University of Essex

August 2026

1.Introduction

Financial transaction data is among the most sensitive domains in data analysis. As e-commerce platforms drive the growth of online payments, the volume of transactional data and, consequently, the incidence of fraud have risen accordingly, particularly fraud committed with stolen credit cards. A fraudulent transaction differs in behaviour from those of the legitimate cardholder, and that difference forms the basis for automated fraud detection [4]. However, building such systems is challenging due to the high number of features, business-specific constraints, and the imbalance amount of the cases in historical data [6]. The problem carries real economic weight: undetected fraud imposes losses on both banks and customers, while excessive false alarms erode customer trust. This report applies four machine learning models to an open dataset of 48 hours of transactions to detect fraudulent cases under severe class imbalance, following a pipeline of scaling, training, validation, threshold selection, and final test evaluation using PR-AUC as the primary metric as it helps for the selection of the best model, model hyperparameters, or sampling rates [2].

2.Dataset description and class imbalance

The dataset [1] was downloaded in full set instead of using the subset of 50,000 records. The below table 1 reports the analysis done:

Observations	284807
Features	30
Features name	"Time","V1","V2","V3","V4","V5","V6","V7","V8","V9","V10","V11","V12","V13","V14","V15","V16","V17","V18","V19","V20","V21","V22","V23","V24","V25","V26","V27","V28","Amount","Class"
Missing values	0
Class	0 284315 no fraud 1 492 fraud
Fraud rate	0.001727485630620034

Table 1—Resume data

The transactions made by credit cards are from September 2013 by European cardholders in two days. The table confirms 492 frauds out of 284,807 transactions. The dataset is highly unbalanced as the positive class (frauds) account for 0.172% of all transactions. There are no missing values to handle. Features V1-V28 are the principal components obtained with PCA, except for 'Time' and 'Amount'. Unfortunately, due to confidentiality the original features and their meaning were not disclosed. 'Time' contains the seconds elapsed between each transaction and the first transaction in the dataset. 'Amount' is the transaction amount, and 'Class' is the response variable where 1 = fraud and 0 =otherwise.

Following the dataset providers' recommendation, the Area Under the Precision-Recall Curve (PR-AUC) is adopted as the primary metric, since confusion-matrix accuracy is not meaningful under severe imbalance. ROCAUC is considered the secondary measure since, in case of serious class imbalance, it can be high despite poor precision for the minority class (fraud). The analysis of the confusion matrix will help to understand the true positives, false negatives, and especially false positives, which are the legitimate users marked falsely.

3. Preprocessing and Splitting

The dataset is split into 60% training, 15% validation, and 25% test sets. The table 2 report fraud counts and percentage in each split.

Split	N	Frauds	Rate
Train	170,884	295	0.1726%
Validation	42,721	74	0.1732%
Test	71,202	123	0.1727%
Total	284,807	492	0.1727%

Table 2— Data splits

Two splitting strategies were considered: a chronological on Time or stratified on the fraud label. A chronological mimics deployment by predicting tomorrow's fraud using yesterday's data. In contrast, a stratified random split shuffles everything, so each split has the same fraud percentage. However, stratification creates a mismatch in time: the model can train on hour 40 and test on hour 12. In deployment this is not possible, as the model must be trained on the past. A chronological split avoids it by respecting the temporal order, training on roughly the first 29 hours and testing on the last 12. However, with only 492 frauds spread unevenly across 48 hours, a chronological split risks leaving some temporal windows with too few or no fraud cases, producing unstable precision and recall estimates. Stratification was therefore selected, as it guarantees a proportional fraud representation in every split (Table 2), with a fixed random state (42) ensuring reproducibility. As a robustness check, the pipeline was re-run under the rejected chronological split. Random Forest reached a PR-AUC of 0.810 on the temporal test window (94 frauds), against 0.839 under stratification. The conclusion is unchanged, and the modest drop is consistent with the temporal instability that motivated the stratified choice.

Feature engineering focused on Time and Amount. Regarding Amount, it was noted that error metrics such as F1 penalise a missed £4 fraud identically to a missed £3,000 fraud. Amount was included, but amount-dependent costs are left as future work. The raw value of "Time" was converted into an hour-of day (0-23) to expose any daily rhythm in fraud. Exploratory analysis on a copy of the training set alone revealed the fraud rate is far from uniform across the day: it spikes to roughly 1.9% at 02:00 and 1.2% at 04:00, against a baseline of 0.17%, making fraud seven to eleven times more likely in the early-morning hours. This trend appears logical since legitimate traffic drops during the night (the number of transactions in hour 02:00 is approximately 2,000 in comparison with 10,000 in midday), which leads to a rise in the proportion of fraud cases. Counting only the number of cases would be erroneous since hour 11:00 contains 33 fraud cases and only a fraud rate of 0.32% due to being one of the busiest hours. Therefore, fraud rate became the basis for the decision. This study considered only the training data set.

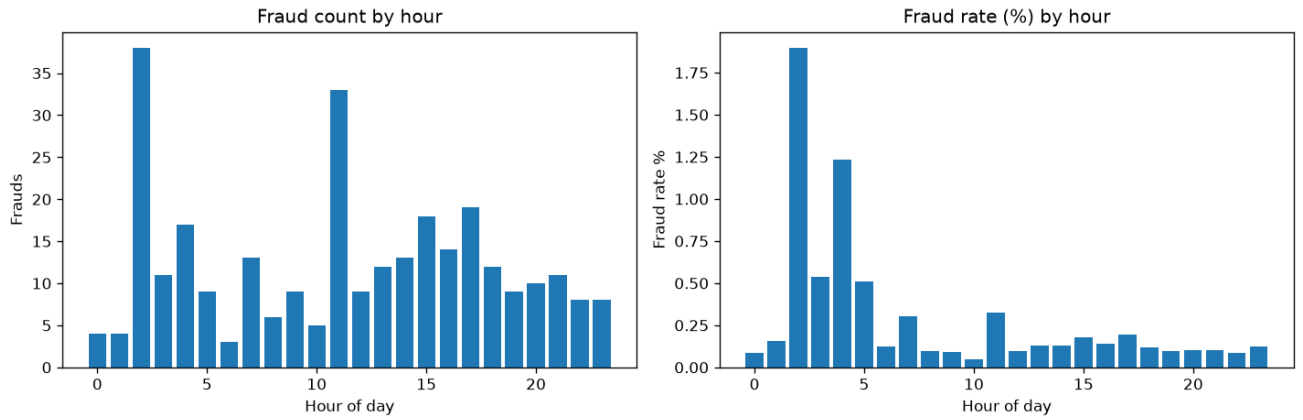


Fig.1. Fraud count and rate by hour

After engineering Hour, the two options of retaining "Time + Hour" or "Hour" alone were compared; Time was dropped, as its removal changed Random Forest PR-AUC only marginally (0.832 without Time versus 0.828 with it), confirming it carried little predictive signal beyond the daily cycle already captured by Hour. Finally, all features were scaled for the models that require it —Logistic Regression, SVM, and the Neural Network — while Random Forest used the unscaled features, since tree-based models are insensitive to scale and scaling would only obscure the feature-importance interpretation. The scaler was fitted on the training set alone and applied to validation and test, preventing information leakage.

4.Methodology & Models

The imbalance was addressed by comparing class weights against resampling (SMOTE) on the training set, together with threshold adjustment. Each approach has limitations, explained in Section 7. Aware of these costs, the comparison between class weights and SMOTE was run on the first model, Logistic Regression, using the training set only and a threshold of 0.5 on validation.

Logistic Regression was trained with L2 regularisation. Its fitted coefficients can be inspected, but they carry no direct business meaning: features V1–V28 are anonymised PCA components, so a large coefficient shows that a component is predictive without revealing what real-world quantity it represents. The largest coefficients in absolute value were V14, V10, V4, and V17, broadly consistent with the components the Random Forest later ranked as most important. This agreement across two very different models suggests the signal is real, not an artefact of one learner. Still, the caveat stands: these components predict fraud without explaining it. Also, output thresholding is considering a more beneficial technique as it does not modify existing learners and can be performed on any learner that provides probability scores [2]. To decide among them, the confusion matrix was read directly, not just the summary numbers ([[TN, FP], [FN, TP]]), alongside the precision-recall shape and PR-AUC at threshold. The results are shown below (Table 3).

Strategy	Precision	Recall	F1	ROC-AUC	PR-AUC
Class weights	0.065	0.892	0.122	0.981	0.712
SMOTE	0.060	0.892	0.112	0.978	0.711

Table 3 — Imbalance strategy comparison (Logistic Regression, validation 0.5)

Class weights caught 66 of 74 and missed 8, raising 944 false alarms. SMOTE caught 66 of 74 frauds and missed 8, but raised 1,034 false alarms. The PR-AUC was 0.712 versus 0.711. Class weights were marginally ahead on precision, recall, F1, and ROC-AUC. SMOTE expanded 295 real frauds to 170,589 by inventing 170,294 synthetic cases. Overall, class weights slightly outperformed SMOTE on the dataset. Class weights were carried forward for all models. All hyperparameters and thresholds were selected on the training and validation sets only. The test set was never touched until the final evaluation. Logistic Regression used L2 regularisation with the default strength ($C = 1$) and balanced class weights. Linear SVM was trained with balanced class weights on the scaled features and wrapped in probability calibration, since SVMs do not output probabilities natively and calibrated scores are required for threshold analysis. A grid search over $C \in \{0.01, 0.1, 1, 10\}$ on validation PR-AUC showed the result was insensitive to regularisation strength (0.698–0.699), with $C=10$ selected. Random Forest used 200 trees with balanced class weights on the unscaled features. Deeper tuning of tree depth was not pursued: validation PR-AUC was already stable and the validation/test gap (Section 5) shows no overfitting. The Neural Network is a small Keras model (two hidden layers of 32 and 16 units, ReLU, sigmoid output) with dropout of 0.3 and early stopping on validation PR-AUC, restoring the best weights. The deliberately small architecture is itself an overfitting control, following the brief.

The false-alarm cost is still high. At threshold 0.5, class weights raise 944 false alarms to catch 66 frauds. Under the scenario in real banking, 944 good customers are marked to uncover 66 frauds, which is an expensive operation. For that reason, the threshold was reconsidered based on the report's operational cost consideration rather than money loss. In case it is set at 0.5, the algorithm would be designed to mark almost all cases. What comes next is to modify the threshold and produce a precision recall graph to see how different thresholds affect FP and false negatives FN.

5.Results & Evaluation

This section reports the performance of the four models and Precision-Recall curve fig.2. Table 4 compares them on the validation set at the default threshold of 0.5, using class weights. Random Forest is the strongest model, with a PR-AUC of 0.832. Logistic Regression follows with 0.712, then Linear SVM with 0.699, and the Neural Network with 0.681. By ROC-AUC, Logistic Regression scores highest (0.981), but by PR-AUC, Random Forest wins. It shows why PR-AUC is the correct metric under severe imbalance.

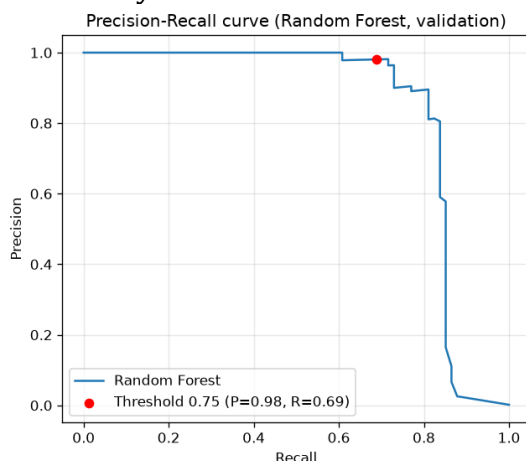


Fig. 2. PR curve shows the same story as a shape: precision holds $\sim 0.9+$ up to about 0.75 recall, then falls off a cliff. The "elbow" around Recall 0.75–0.85 is where sensible operating points live.

Model	Precision	Recall	F1	ROC-AUC	PR-AUC	Configuration
Random Forest	0.902	0.743	0.815	0.935	0.832	Class weights, unscaled features
Logistic Regression	0.065	0.892	0.122	0.981	0.712	Class weights, scaled, L2
Linear SVM	0.872	0.554	0.678	0.979	0.699	Class weights, scaled, calibrated
Neural Network	0.077	0.851	0.142	0.942	0.675	Class weights, scaled, dropout, early stopping

Table 4 - Model comparison (validation, threshold 0.5, class weights)

Threshold analysis for Random Forest using validation set is presented in Table 5 with increase of the threshold, precision grows and recall decreases. When threshold equals to 0.5, precision is 0.902, and recall is 0.743. For threshold of 0.75, precision becomes 0.981 and recall becomes 0.689. It is seen from Table 6 that when threshold equals 0.75, only Random Forest has both precision and recall.

Threshold	Precision	Recall
0.10	0.674	0.838
0.30	0.833	0.811
0.365 (F1-optimal)	0.896	0.811
0.50	0.902	0.743
0.75 (chosen)	0.981	0.689
0.90	1.000	0.541
0.99	1.000	0.149

Table 5— Random Forest threshold analysis (validation)

Model	Precision	Recall	F1
Random Forest	0.981	0.689	0.810
Logistic Regression	0.121	0.878	0.213
Linear SVM	0.864	0.514	0.644
Neural Network	0.570	0.824	0.674

Table 6 — Model comparison (validation, threshold 0.75)

Table 7 presents the final test results. A random classifier achieves a PR-AUC equal to the fraud rate (0.0017); RF PR-AUC (0.839), close to its validation value (0.832). The small gap indicates the model generalises well and did not overfit. At the chosen threshold of 0.75, Random Forest raises only two false positives across 71,079 legitimate transactions, catching 82 of 123 frauds. It matches the operational goal of minimising false alarms.

Model	Threshold	Precision	Recall	F1	ROC-AUC	PR-AUC
Random Forest	0.50	0.913	0.764	0.832	0.956	0.839
Random Forest	0.75	0.976	0.667	0.792	0.956	0.839
Logistic Regression	0.50	0.063	0.886	0.117	0.976	0.703
Logistic Regression	0.75	0.123	0.870	0.216	0.976	0.703

Table 7 — Model comparison (Test, threshold 0.75 vs 0.50, class weights)

6. Discussion

Precision and recall are the two cost types involved in detecting fraud that differ economically. False negatives are an example of undetected fraud, and this will result in financial losses for the bank. On the other hand, a false positive represents a legal transaction that is incorrectly marked, hence leading to frustration on the customer's side and increased work for the analysts, along with the possibility of losing the client because the card might be blocked unjustifiably. While, false negatives should be costlier since money is lost, this report will favor precision instead. In the competitive world we live in today, too many false positives lead to overload of the customer support services, which in turn leads to losing legitimate clients. This tradeoff can be shown using the threshold sweep of the random forest model; precision rises from 0.674 at a threshold of 0.10 to 0.981 at 0.75, while recall goes down from 0.838 to 0.689.

In *model comparison*, Random Forest is the best model clearly ahead of the others. The

ranking depends on the metric. By ROC-AUC, Logistic Regression looks better (0.976 on test). By PR-AUC, Random Forest wins. The disagreement is itself evidence that PR-AUC is the right metric here. ROC-AUC is reported for completeness, but PR-AUC is used as the primary metric because the fraud class represents only 0.1727% of transactions. In this setting, a model can obtain a high ROC-AUC while still producing an operationally poor precision-recall trade-off on the rare positive class. PR-AUC therefore provides a more informative view of fraud-detection performance. Logistic Regression has the best recall (0.892) and the best ROC-AUC (0.981 on validation, 0.976 on test). Random Forest has the best precision (0.902), F1 (0.815) and PR-AUC (0.832) on validation, and holds that lead on test. Thus, the answer to whether the best ROC-AUC model is also best elsewhere is no, and that is precisely why the headline metric matters.

Random Forest has great generalization due to its test PR-AUC (0.839) almost matches its validation PR-AUC (0.832). The small gap shows the model did not overfit. Overfitting was controlled during training, not just checked afterwards. Dropout and early stopping were used on the neural network, where the process was stopped once validation PR-AUC stopped improving and reverted back to the weights that produced the optimal result. L2 regularization was used for the Logistic Regression model to regulate the magnitude of the coefficients. Weighting was used instead of resampling because of the risk of introducing artificial structure through resampling. The good match in validation and test performances of all the models suggests that regularization and weighting worked well. Two further tests confirm this claim. A bootstrap procedure (1,000 resamples) produces a 95% CI for test PR-AUC of [0.768, 0.904], with a mean of 0.839. The CI is relatively wide because of only 123 fraud cases in the test set. Both figures still sit far above the 0.0017 a random classifier would achieve, which is the reference point for PR-AUC under this imbalance.

Class Imbalance is addressed by comparing class weights against SMOTE, applied to the training set only. Class weights slightly outperformed SMOTE (validation PR-AUC 0.712 versus 0.711). They are also simpler, faster, and left the training data unaltered. Class weights were carried forward for all models. Expanding 295 real frauds to 170,589 by generating 170,294 synthetic cases added little real signal in the thirty-dimensional PCA space.

The Random Forest feature importances rank V14, V10, V4, and V17 as most predictive. The interpretability of results is limited. V1-V28 are anonymised PCA components, and their original recipe was never released. It is possible to say which components predict fraud. It is not possible to say what real-world quantity each one represents. Feature importance shows where the signal lies, not the business behaviour behind it. It constrains any operational insight the bank could draw from the model.

The default threshold of 0.5 and the chosen threshold of 0.75 were both evaluated, along with the F1-optimal threshold of 0.365. A threshold of 0.75 was chosen for operational reasons, not statistical ones. On the test set it gives a precision of 0.976. It raises only two false positives across 71,079 legitimate transactions. The cost is missing about 33% of fraud (recall 0.667). This is a deliberate business decision. 0.75 is not the statistically optimal point, which is 0.365. This is a concrete decision-making example where there is no available monetary cost matrix. Future work should quantify the monetary cost of the missed fraud to recalibrate the selected threshold against real deployment costs.

7.Limitations & Future Work

Some of the limitations are attached to the results obtained in this study. For instance, the early morning fraud distribution found through the exploratory analysis can be caused by the real fraudsters or a consequence of one of the bursts. The limitation should be viewed as indicative because only two daily bursts have been detected. Every mitigation method applied in order to tackle the issue of data imbalance brings some costs. If we weight transactions with their amount, this solution can help solve problems at the moment but fail to detect problems in the future. SMOTE is the method with dual benefits. It allows detecting patterns of the minority class when synthetic points are created close to the real ones but it creates false patterns otherwise. On the other hand, the creation of synthetic frauds based on only a few real frauds which create a bell-shaped distribution similar to the one of real frauds can be correct but it should be done cautiously since some interpolated points can refer to the frauds which do not occur in the deployment process. One more limitation is the increase in dataset size [2]. SMOTE proved to be inefficient in the experiment due to known limitations of linear interpolation in high-dimensional spaces [3].

Threshold adjustment depends entirely on the business goal. If the target is to recover most fraud by value, a lower threshold flags more transactions as fraud but generates many false positives and dissatisfied customers. A higher threshold reduces false alarms but leaves larger sums undetected. The analysis also inherits structural limits from the dataset. The anonymised PCA features block any business-level interpretation of the model. There is no customer-level history and no merchant information, which production fraud systems rely on heavily. Only 48 hours of transactions from 2013 are available, which raises the risk of dataset shift on current data.

Future work includes using a simulated credit card transaction dataset containing legitimate and fraud transactions from more than a year, covering credit cards of 1000 customers doing transactions with 800 merchants [5]. Additionally, quantify the monetary cost of missed fraud to guide threshold selection, and test distribution-aware SMOTE variants such as Borderline-SMOTE or ADASYN [3] against the class-weights baseline used here.

References

1. Kaggle: credit card fraud detection. <https://www.kaggle.com/mlg-ulb/creditcardfraud>.
2. Leevy, J.L., Johnson, J.M., Hancock, J. et al. Threshold optimization and random undersampling for imbalanced credit card data. *J Big Data* 10, 58 (2023). <https://doi.org/10.1186/s40537-023-00738-z>
3. Idwan, S., Etaiwi, W., Rafayia, H. et al. A comprehensive review of statistical variants and enhancements of SMOTE oversampling method. *Int J Data Sci Anal* 20, 6887–6904 (2025). <https://doi.org/10.1007/s41060-025-00878-w>
4. Stamile, C., Marzullo, A., Deusebio, E. (2025). *Graph Machine Learning* (2nd ed.). Packt Publishing
5. Kaggle : Credit Card Transactions Fraud Detection Dataset
<https://www.kaggle.com/datasets/kartik2112/fraud-detection/data>
6. Khan, S., Alourani, A., Mishra, B., Ali, A., & Kamal, M. (2022). Developing a credit card fraud detection model using machine learning approaches. *International Journal of Advanced Computer Science and Applications*, 13(3) doi:<https://doi.org/10.14569/IJACSA.2022.0130350>